LOGISTIC REGRESSION

1. What is Logistic Regression?

Logistic regression is a **supervised machine learning algorithm** used for **classification tasks**. Despite its name containing the word "regression", it is primarily a classifier — it predicts which **category or class** a data point belongs to.

The most common use case is **binary classification** — where the output can be one of only two classes:

- **Class 1 (Positive):** e.g., Spam, Disease Present, Fraud, Yes
- **Class 0 (Negative):** e.g., Not Spam, Healthy, Legitimate, No

Real-World Examples:

- Email spam detection (Spam vs. Not Spam)
- Medical diagnosis (Disease vs. No Disease)
- Credit scoring (Default vs. No Default)
- Image classification (Cat vs. Dog)

> KEY INSIGHT: Logistic Regression does NOT directly predict a class label. Instead, it predicts the PROBABILITY that a data point belongs to Class 1. Then a threshold (usually 0.5) is applied to convert that probability into a final class label.

Why not use Linear Regression for classification?

Linear Regression predicts values from $-\infty$ to $+\infty$. But probabilities must lie between 0 and 1. If we used linear regression for classification:

- Predicted values could be negative (e.g., −0.3) or greater than 1 (e.g., 1.7) — not valid probabilities
- It is highly sensitive to outliers, making the boundary unstable

Logistic Regression solves this by applying the Sigmoid function to the linear output.

---

2. The Sigmoid Function

The **sigmoid function** (also called the *logistic function*) is the heart of logistic regression. It squashes any real number into the range **(0, 1)**, making it perfect for representing probabilities.

The Formula

```
σ(z)  =  1 / (1 + e^(-z))
```

Where:

σ(z)	The sigmoid function output — a probability between 0 and 1
z	The input: the linear combination of features (also called the "logit")
e	Euler's number ≈ 2.718 (the base of natural logarithm)
e^(−z)	The exponential of negative z

VISUAL: The sigmoid curve is S-shaped. It starts near 0 for very negative inputs, crosses 0.5 at z=0, and approaches 1 for very positive inputs. The steepest change happens around z=0, which is where the decision boundary lies.

3. The Logistic Regression Equation

Logistic regression works in two steps:

Step 1 — Compute the Linear Combination (z)

First, a weighted sum of the input features is computed — exactly like linear regression:

$$z = \beta_0 + \beta_1 \cdot x_1 + \beta_2 \cdot x_2 + \beta_3 \cdot x_3 + \dots + \beta_n \cdot x_n$$

z	The raw linear output (called the logit or log-odds)
β_0	Bias / Intercept — the baseline log-odds when all features = 0
$\beta_1, \beta_2, \dots, \beta_n$	Weights / Coefficients — one per feature, learned during training
$x_1, x_2, \dots, x_n$	Input feature values for a data point
n	Number of features

Step 2 — Apply the Sigmoid to Get Probability

The linear output z is passed through the sigmoid function to produce a probability:

$$P(y=1 \mid x) = \sigma(z) = 1 / (1 + e^{(-z)})$$

Step 3 — Apply the Threshold to Get Class Label

The predicted probability is converted into a final class label using a threshold (typically 0.5):

$$\hat{y} = 1 \text{ if } P(y=1|x) \geq 0.5, \quad \text{else } \hat{y} = 0$$

4. Training: Maximum Likelihood & Cross-Entropy Loss

Logistic regression is trained by finding the weights β that make the observed data most likely. This is called Maximum Likelihood Estimation (MLE).

Binary Cross-Entropy Loss (Log Loss)

The loss function used is called Binary Cross-Entropy. It measures how well the predicted probabilities match the actual class labels:

$$L = -(1/n) \times \Sigma_i \left[y_i \cdot \log(\hat{y}_i) + (1-y_i) \cdot \log(1-\hat{y}_i) \right]$$

n	Total number of training samples
y_i	True label for sample i (either 0 or 1)
$\hat{y}_i$	Predicted probability (from sigmoid) for sample i
log	Natural logarithm (ln)
Σ	Sum over all training samples

Understanding the Cross-Entropy Formula

Case 1: True label $y_i = 1$ (Actual Positive)

The loss simplifies to: $L = -\log(\hat{y}_i)$

- If $\hat{y}_i \approx 1.0$ (correct and confident): loss ≈ 0 (good!)
- If $\hat{y}_i \approx 0.5$ (uncertain): loss ≈ 0.69 (moderate penalty)
- If $\hat{y}_i \approx 0.0$ (wrong and confident): loss → ∞ (huge penalty!)

Case 2: True label $y_i = 0$ (Actual Negative)

The loss simplifies to: $L = -\log(1 - \hat{y}_i)$

- If $\hat{y}_i \approx 0.0$ (correct and confident): loss ≈ 0 (good!)
- If $\hat{y}_i \approx 0.5$ (uncertain): loss ≈ 0.69 (moderate penalty)
- If $\hat{y}_i \approx 1.0$ (wrong and confident): loss → ∞ (huge penalty!)

WHY CROSS-ENTROPY? It is derived directly from Maximum Likelihood Estimation. Minimising cross-entropy = maximising the likelihood of the training data. It penalises confident wrong predictions very heavily (loss → ∞), which forces the model to calibrate its probabilities well. The loss surface is convex — guaranteed single global minimum.

Optimization

There is no closed-form solution. The model is trained using iterative optimization algorithms:

- **Gradient Descent:** Iteratively adjust weights in the direction that reduces loss
- **Stochastic Gradient Descent (SGD):** Uses one sample at a time; faster for large datasets
- **L-BFGS:** A more advanced quasi-Newton method; often used for smaller datasets

Gradient of loss w.r.t. weights:

$$\partial L/\partial \beta = (1/n) \times X^T \times (\hat{y} - y)$$

This has the same clean form as linear regression's gradient — a key advantage of the logistic + cross-entropy combination.

5. Decision Threshold

The decision threshold is the probability cutoff used to convert a predicted probability into a binary class label. By default it is 0.5.

```
Predict Class 1  if  P(y=1|x) ≥ threshold Predict Class 0  if
                     P(y=1|x) < threshold
```

Why Tune the Threshold?

The default threshold of 0.5 is not always optimal. Depending on the business context:

Scenario	Preferred Threshold	Reason
Medical diagnosis (cancer)	Lower (e.g. 0.3)	Missing a real case (false negative) is dangerous. Better to flag more positives.
Spam detection	Higher (e.g. 0.7)	Blocking legitimate emails (false positives) is costly. Be conservative.
Fraud detection	Lower (e.g. 0.2)	Missing fraud has severe financial consequences.
Recommendation system	Around 0.5	Balanced error costs.

Threshold Effect on Precision and Recall

- **Lower threshold** → More positives predicted → Higher Recall, Lower Precision
- **Higher threshold** → Fewer positives predicted → Higher Precision, Lower Recall

> The ROC curve (Section 13) shows model performance across ALL possible thresholds simultaneously — it helps you choose the best threshold for your use case.

---

6. Confusion Matrix

A confusion matrix is a table that summarizes the performance of a classification model by comparing actual vs. predicted labels. It is the foundation for computing all classification metrics.

		Predicted	
		Positive (1)	**Negative (0)**
Actual	**Positive (1)**	**TP (True Positive)**	**FN (False Negative)**
	Negative (0)	**FP (False Positive)**	**TN (True Negative)**

The Four Cells Explained

TP — True Positive	Model said Positive (1) AND it was actually Positive (1). Correct prediction of a positive case.
TN — True Negative	Model said Negative (0) AND it was actually Negative (0). Correct prediction of a negative case.
FP — False Positive (Type I Error)	Model said Positive (1) BUT it was actually Negative (0). Also called a "False Alarm".
FN — False Negative (Type II Error)	Model said Negative (0) BUT it was actually Positive (1). Also called a "Miss".

Medical Example

Suppose we test 1000 patients for a disease using a logistic regression model:

	Predicted: Disease (1)	**Predicted: Healthy (0)**	**Row Total**
Actual: Disease (1)	TP = 80	FN = 20	100 actual positive
Actual: Healthy (0)	FP = 30	TN = 870	900 actual negative
Column Total	110 predicted positive	890 predicted negative	1000 total

From this confusion matrix: TP=80, TN=870, FP=30, FN=20 All of the metrics below (Accuracy, Precision, Recall, F1, Specificity) are calculated directly from these four numbers.

---

7. Accuracy

Accuracy is the most intuitive metric — the proportion of all predictions that were correct.

```
Accuracy = (TP + TN) / (TP + TN + FP + FN)
```

Using our medical example: Accuracy = (80 + 870) / 1000 = 950/1000 = 0.95 = 95%

Interpretation

- 95% of all predictions (both positive and negative) were correct
- Easy to understand and communicate to non-technical stakeholders

When Accuracy is Misleading: The Imbalanced Class Problem

Accuracy can be very misleading when the data is imbalanced (one class is much more common than the other):

EXAMPLE: Suppose only 1% of transactions are fraudulent. A naive model that ALWAYS predicts 'Not Fraud' achieves: Accuracy = 99% — but detects ZERO fraudulent transactions! This is why accuracy alone is never enough for imbalanced classification problems. Use Precision, Recall, and F1 instead.

8. Precision

Precision answers: "Of all the cases I predicted as Positive, how many were actually Positive?"

```
Precision = TP / (TP + FP)
```

Using our example: Precision = 80 / (80 + 30) = 80/110 ≈ 0.727 = 72.7%

Interpretation

- Out of 110 patients our model flagged as having the disease, 80 actually had it
- 30 were false alarms (FP) — unnecessarily worried healthy patients

When Precision Matters Most

High precision is critical when the cost of a False Positive is high:

- Spam filtering: Marking legitimate email as spam is very annoying
- Drug approval: Approving an ineffective drug has consequences
- Legal decisions: Falsely accusing an innocent person

> Precision focuses ONLY on what the model claims are positive predictions. It ignores true negatives (TN) and false negatives (FN) completely.

9. Recall (Sensitivity / True Positive Rate)

Recall answers: "Of all the actual Positives, how many did I correctly identify?"

```
Recall = TP / (TP + FN)
```

Using our example: Recall = 80 / (80 + 20) = 80/100 = 0.80 = 80%

Interpretation

- Out of 100 patients who actually had the disease, our model correctly identified 80
- We missed 20 sick patients (FN) — they left without treatment

When Recall Matters Most

High recall is critical when the cost of a False Negative is high:

- Cancer screening: Missing a real cancer case is life-threatening
- Fraud detection: Missing actual fraud costs money
- Safety systems: Missing a dangerous condition causes accidents

> Recall is also called SENSITIVITY or TRUE POSITIVE RATE (TPR). It measures how well the model 'catches' all the actual positives. High recall means: fewer things slip through the net.

10. F1 Score

The F1 Score is the harmonic mean of Precision and Recall. It balances both metrics into a single score, especially useful when classes are imbalanced.

```
F1 = 2 × (Precision × Recall) / (Precision + Recall)
```

Using our example: F1 = 2 × (0.727 × 0.80) / (0.727 + 0.80) = 2 × 0.582 / 1.527 ≈ 0.762

Why Harmonic Mean?

The harmonic mean punishes extreme imbalances between precision and recall.

- **β = 0.5 (F0.5):** Precision weighted twice as much as recall

11. Specificity (True Negative Rate)

Specificity answers: "Of all the actual Negatives, how many did I correctly identify as Negative?"

```
Specificity = TN / (TN + FP)
```

Using our example: Specificity = 870 / (870 + 30) = 870/900 = 0.967 = 96.7%

Interpretation

- Out of 900 healthy patients, our model correctly identified 870 as healthy
- 30 were falsely flagged as diseased (FP) — unnecessarily alarmed

> Specificity is the complement of the False Positive Rate (FPR): FPR = FP / (FP + TN) = 1 − Specificity Specificity is used heavily in medical testing and as the x-axis of the ROC curve (as 1 − Specificity).

Sensitivity vs Specificity Trade-off

There is always a trade-off between sensitivity (recall) and specificity:

- Lowering the threshold → Higher Sensitivity, Lower Specificity (catch more positives but more false alarms)
- Raising the threshold → Higher Specificity, Lower Sensitivity (fewer false alarms but miss more real positives)

12. ROC Curve and AUC

The ROC (Receiver Operating Characteristic) curve is a graph that shows the performance of a classification model at all possible thresholds. It plots:

```
X-axis:  FPR = FP / (FP + TN) = 1 - Specificity Y-axis:  TPR = TP /
                    (TP + FN) = Recall / Sensitivity
```

How to Read the ROC Curve

Each point on the ROC curve represents one threshold setting:

- Top-left corner (TPR=1, FPR=0) = Perfect classifier
- Diagonal line (y = x) = Random classifier (coin flip)
- Bottom-right (TPR=0, FPR=1) = Worst possible classifier

AUC — Area Under the Curve

The AUC (Area Under the ROC Curve) summarises the entire ROC curve into a single number between 0 and 1:

AUC Value	Interpretation
AUC = 1.0	Perfect model — correctly classifies all samples with full confidence
AUC = 0.9 – 1.0	Excellent discriminating ability
AUC = 0.8 – 0.9	Good model
AUC = 0.7 – 0.8	Fair model
AUC = 0.6 – 0.7	Poor model
AUC = 0.5	No discriminative ability — equivalent to random guessing
AUC < 0.5	Worse than random (predictions are inverted)

IMPORTANT: AUC measures the model's ability to rank positives above negatives — it is threshold-independent. Physically: AUC = Probability that a randomly chosen positive sample is ranked higher than a randomly chosen negative sample by the model.